Assignment 3:

Sensor Characteristics Report

Conner Inman

AUTONOMOUS ROBOTICS

February 16, 2026

The goal of Assignment 3: Sensor Characteristics Demonstration was to implement sensing mechanisms into our robot and use the data from those sensors to produce autonomous behaviors. This assignment consisted of 4 parts: Active Directionality Sensing required mounting the optical sensor to a servo and writing a program to point the sensor toward the brightest direction. Next was Accuracy Measurements, where we measured the potentiometer angle using a string, then compared it to the direction detected by the light sensor and calculated the root mean square (RMS) error. Then we moved to Part 3, Phototaxis, which required using our light-sensing code and implementing a drive function to move toward the light. This became the most difficult part for our group, as we tended to both overengineer and under engineer different parts of the code. Lastly, the Color Track required removing the light-sensing mechanism and mounting an optical sensor on the bottom of the robot to follow a red, green, and blue track. Overall, the assignment progressed from static sensing, to measurement accuracy, to autonomous navigation, and finally to color-based path following.

Active Directionality Sensing required installing a light sensor that could point toward the brightest light source it detected. We first did this by installing a light sensor for the robot as shown in Fig. 6. With the servo-mounted light sensor assembled, we then wrote the control code. The code we used is shown in Fig. 9. It moved the servo one degree at a time for 100°s (50→ 0 → -50) and would constantly be running a comparison loop. If the detected light is brighter than the current maximum light reading, then change the max brightness to that new light reading and record the degree that light measurement was found on. At the end of the function, the servo returned the angle associated with the maximum light reading. This value was also printed for debugging, and the servo then rotated to the brightest detected angle. This part was straightforward, as I had already planned the scanning logic before class.

For Accuracy Measurement, we used potentiometer readings to determine exactly where the light sensor was pointing and compare that to the detected light direction. For this as shown in Fig. 1 and Fig. 2 we would place the bot in a pre-measured box (for consistency) where we would then place pieces of tape on the table to pre-determine where our light source would be. We would then take the measure of the potentiometer using a piece of string to make sure the light sensor was pointing in the correct direction, then record it by writing it on the tape and on our notes. We would then run the same comparison loop in Fig. 9 and print the results of the potentiometer and light intensity (for record keeping and debugging). These angles were recorded and used in the error equation $(x-y)^2$. As is shown in Fig. 4 and Fig. 5 where the output degree would be plugged into the Y variable and the original degree would be in the X variable. After all, six positions were measured, we summed the squared errors and divided them by six to compute the mean. We found that the small light cone produced the best result, with an RMS error of 2.65 (Fig. 3). Thus, completing our requirement for this part of the assignment. Once we understood the directions, we were able to complete this part quickly.

For the Phototaxis part of the assignment, we were required to have the robot drive towards a light source. If the robot could not find a light source, it wandered randomly until one

was detected (Fig. 10). We began by moving the light sensor to the front of the robot (Fig. 7). This was done to allow us to have a controlled measurement area as now the 100° scan range was mounted to the front center of the robot thus also making movement easier to program. Once this was done, we decided that performing a full servo scan each time would be too slow, so we implemented a faster 360° robot rotation instead. This was all implemented in the function SpinMeRightRound() (Fig .11). When that function was done, we would then run an optimized Scan() function Fig. 12. This function scanned a 100° range and reused the comparison loop from Part 1, recording both light intensity and the corresponding angle. If multiple adjacent readings had identical light values, the robot counted how many occurred in a row and averaged them to center the detected angle. The count was divided by two to center the detected region, and this value was added to the angle estimate. The function then normalized the detected angle relative to the drivetrain heading so the robot could turn accurately. Then it would drive forward. Fig. 13 is the script which would run the code itself.

This was by far the hardest part for us. The code itself proved to be a sticking point since much of the built-in functionality within the interpreter was itself not one hundred percent clear. Additionally, glare from the glossy table surface negatively affected performance. As the robot may see this glare from this finish and perceive it to be brighter than the light itself. This caused the robot to drive toward the brightest perceived object, even when it was not the intended light source.

For the Color Track we were required to have the optical sensor detect contrasting colors and follow the track to the finish. This was done by mounting the optical sensor to the bottom of the robot (Fig. 8) and having a comparison loop (Fig. 15) check what color the sensor discovered. This ran inside a loop that compared detected colors (Fig. 17). For example, if the robot saw red, it turned right. The main challenge was handling transitions between marked track segments and predetermined launch points. We implemented a list that stored the last detected color before reaching carpet. So, if it saw blue → red → carpet, it would see that red was the color before carpet and make a choice from there. This part required extensive trial and error to refine the behavior.

The robot's performance overall was pretty good. Our main challenges were related to implementation and interpretation rather than core concepts. The exact use of built-in functions caused some issues, but we worked through them gradually. The autonomous behavior we used worked quite well as we did iterative testing. We built the robot incrementally, implementing and testing one function at a time. Nothing was completely hard-coded outside of transferring data between functions to make calculations and decisions. There are a few instances where we did have to hard code some behavior, however this was limited to the end of a task and not the entire program. However, the sensors were often inconsistent, especially under changing lighting conditions. While the sensors behaved as designed, the shift from integer to floating-point light values introduced additional complexity. We found that with the sensing of light now being more reliant on floating-point values it caused issues like the aforementioned glare. Instructions in this

and previous projects occasionally left room for interpretation. We were sometimes unclear on setup details and how to better integrate motors and sensors. However, yet again, the struggle has made us a better group and will be important as time goes on.

Appendix

Figure 1. Experimental setup for accuracy measurements, showing the robot positioned inside the predefined measurement area.

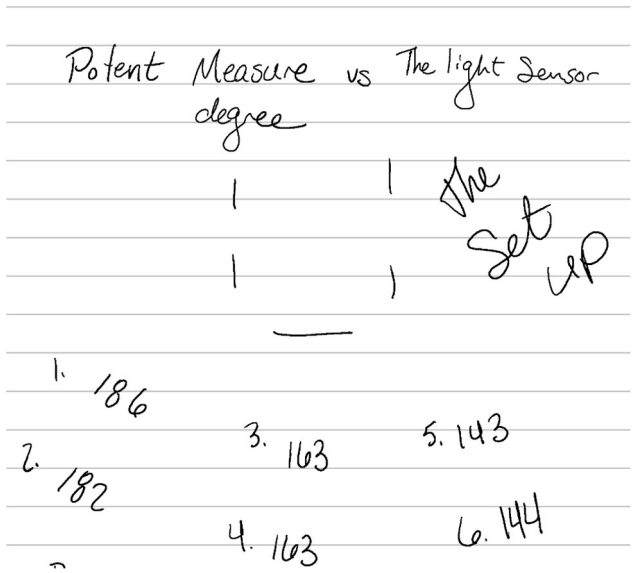

Figure 2. Physical accuracy measurement setup used to ensure consistent sensor positioning and angle testing.

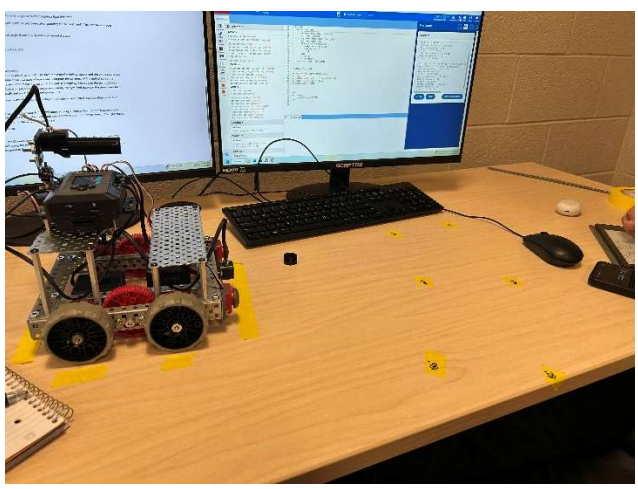

Figure 3. RMS error calculations comparing potentiometer angles (left) and detected scan angles (right) for the no-cone and small-cone conditions.

No Blinder

1. 186 − 183 = 3 = 9
2. 182 − 189 = −7 = 49
3. 163 − 169 = −6 = 36
4. 163 − 162 = 1 = 1
5. 143 − 150 = −7 = 49 + 169
6. 144 − 149 = −5 = 25

169 ÷ 6 = √28.17 = 5.307

Small Blinder

1. 186 − 183 = 3^2 = 9
2. 182 − 178 = 4^2 = 16
3. 163 − 160 = 3^2 = 9
4. 163 − 163 = 0^2 = 0 + 42
5. 143 − 141 = 2^2 = 4
6. 144 − 146 = -2^2 = 4

42 ÷ 6 = √7 = 2.65

Figure 4. RMS error calculations comparing potentiometer angles (left) and detected scan angles (right) for the large-cone condition.

Big Blinder

1. 186 − 191 = -5^2 = 25
2. 182 − 188 = -6^2 = 36
3. 163 − 170 = -7^2 = 49
4. 163 − 169 = -6^2 = 36 + 259
5. 143 − 151 = -8^2 = 64
6. 144 − 151 = -7^2 = 49

259 ÷ 6 = √43.17 = 6.57

Figure 5. Diagram of the color track used to plan navigation and decision logic.

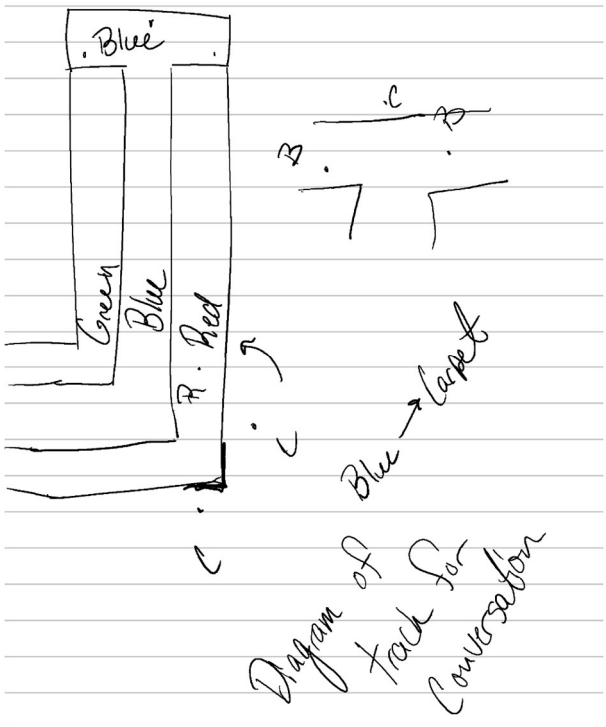

Figure 6. Design sketch of the servo-mounted light-sensing mechanism used for directional scanning.

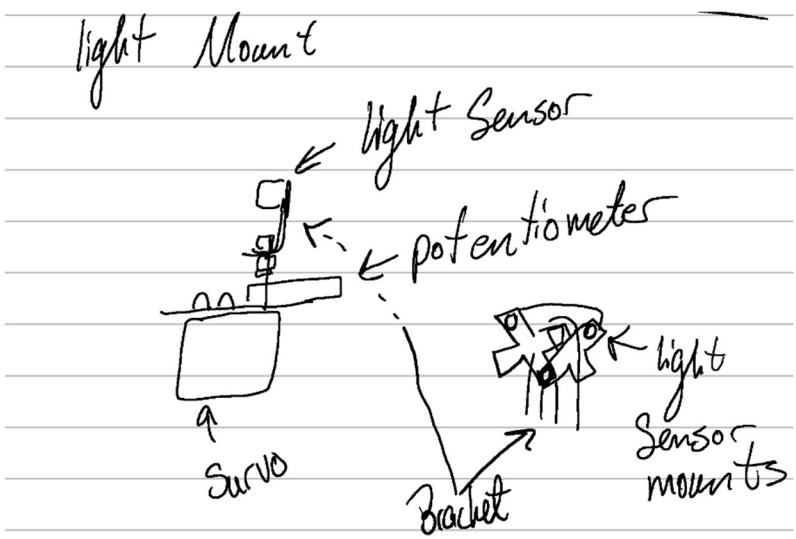

Figure 7. Front-mounted light sensor configuration used during phototaxis testing.

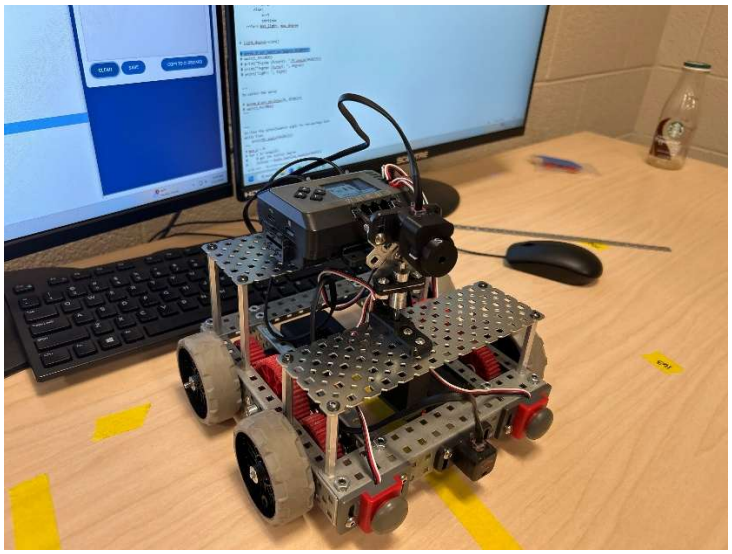

Figure 8. Downward-facing optical sensor mounted underneath the robot for color track detection.

Figure 9. Servo scanning function used for Parts 1–3, including print statements for measurement and debugging.

```
#
def scan():
    x=-50
    max_light=0
    max_degree=0

    servo_d.set_position(-50)

    while (x <= 50):
        cur_light = optical_6.brightness()
        servo_d.set_position(x,DEGREES)
        wait(0.05,SECONDS)
        # print("current light:",cur_light)
        # print("degree:",x)
        if cur_light > max_light:
            max_light = cur_light
            max_degree = x
            x+=1
        else:
            x+=1
            continue
    return max_light, max_degree


# light,degree=scan()

# servo_d.set_position(degree,DEGREES)
# wait(1,SECONDS)
# print("Degree (Potent): ",PC.angle(DEGREES))
# print("Degree (survo): ", degree)
# print("Light: ", light)

"""
to center the servo

# servo_d.set_position(0, DEGREES)
# wait(1,SECONDS)
"""
```

Figure 10. Function that rotates the robot until a light source is detected.

```
def MeBlind(light):
    if (light < 0.1):
        drivetrain.turn(LEFT)
        wait(ran.randint(0.5,5), SECONDS)
```

Figure 11. Initial light localization function used to estimate the general direction of the light source.

```
def spinMeRightRound():
    drivetrain.turn_to_heading(0, DEGREES)
    wait(0.5, SECONDS)

    max_light = 0
    max_degree = 0

    for _ in range(12):  # 12 turns Ã— 30Â° = 360Â°
        drivetrain.turn_for(RIGHT, 30, DEGREES)
        wait(0.2, SECONDS)

        cur_light = optical_6.brightness()
        cur_heading = drivetrain.heading(DEGREES)

        print("Heading:", cur_heading, "Light:", cur_light)

        if cur_light > max_light:
            max_light = cur_light
            max_degree = cur_heading

    return max_light, max_degree
```

Figure 12. Fine-tuned scanning function combining servo motion and light sensing to determine a precise heading.

```
def scan():
    max_light = 0
    max_angle = -30
    count_samel = 1
    for angle in range(-30, 30):
        servo_d.set_position(angle, DEGREES)
        wait(0.1, SECONDS)
        cur_light = optical_6.brightness()

        print("Servo Angle:", angle, "Light:", cur_light)

        if cur_light > max_light and cur_light != max_light:
            max_light = cur_light
            max_angle = angle
            count_samel = 1
        elif cur_light == max_light:
            count_samel += 1

    count_samel /= 2
    max_angle = max_angle + count_samel
    # Convert servo angle to robot heading
    robot_heading = drivetrain.heading(DEGREES)
    target_heading = (robot_heading - max_angle) % 360

    return max_light, target_heading
```

Figure 13. Main control script integrating sensing and motion functions for autonomous behavior.

```
ight, degree = spinMeRightRound()

hile True:
    if distance_10.object_distance(INCHES) >= 1:
        drivetrain.turn_to_heading(degree, DEGREES)
        wait(1, SECONDS)

        max_light, degree = scan()  # update degree here
        servo_d.set_position(degree,DEGREES)
        wait(1,SECONDS)


        drivetrain.turn_to_heading(degree-1, DEGREES)
        wait(1,SECONDS)

        drivetrain.drive_for(FORWARD, 20, INCHES)
        wait(1,SECONDS)


    else:
        drivetrain.drive_for(REVERSE, 3, INCHES)
        drivetrain.drive_for(FORWARD, 2, INCHES)
        drivetrain.turn_to_heading(ran.randint(0,360), DEGREES)
        wait(0.5, SECONDS)
```

Figure 14. Configuration and initialization code for motors, sensors, and drivetrain settings.

```
# ----------------------------
# Device Configuration
# ----------------------------

brain = Brain()

brain_inertial = Inertial()
optical = Optical(Ports.PORT6)

left_motor = Motor(Ports.PORT1, False)
right_motor = Motor(Ports.PORT5, True)

drivetrain = SmartDrive(
    left_motor,
    right_motor,
    brain_inertial,
    219.44,   # wheel travel (mm)
    320,      # track width (mm)
    40,       # wheelbase
    MM,
    1
)

# ----------------------------
# Calibration
# ----------------------------

def calibrate_drivetrain():
    brain.screen.print("Calibrating Inertial...")
    brain_inertial.calibrate()
    while brain_inertial.is_calibrating():
        wait(25, MSEC)
    brain.screen.clear_screen()

calibrate_drivetrain()

# ----------------------------
# Settings
# ----------------------------

drive_speed = 7.5   # %
turn_speed = 15     # %
turn_amount = 4     # degrees

drivetrain.set_drive_velocity(drive_speed, PERCENT)
drivetrain.set_turn_velocity(turn_speed, PERCENT)
drivetrain.set_stopping(BRAKE)

optical.set_light_power(100, PERCENT)
optical.set_light(LedStateType.ON)
```

Figure 15. Hue comparison logic used to classify detected colors on the track.

```
# ----------------------------
# Color Ranges (Hue)
# ----------------------------

RED_MIN, RED_MAX = 0, 15
GREEN_MIN, GREEN_MAX = 55, 80
BLUE_MIN, BLUE_MAX = 230, 260

def read_hue():
    return optical.hue()

def is_red(h):
    return RED_MIN <= h <= RED_MAX

def is_green(h):
    return GREEN_MIN <= h <= GREEN_MAX

def is_blue(h):
    return BLUE_MIN <= h <= BLUE_MAX

def is_carpet(h):
    return 20 <= h <= 30
```

Figure 16. Movement helper functions defining forward motion and turning behaviors.

```
# ----------------------------
# Movement Helpers
# ----------------------------

def drive_forward():
    drivetrain.drive(FORWARD)

def turn_left():
    drivetrain.turn_for(LEFT, turn_amount, DEGREES, wait=True)

def turn_right():
    drivetrain.turn_for(RIGHT, turn_amount, DEGREES, wait=True)
```

Figure 17. Main execution loop used to follow the color track and handle transitions between segments.

```python
# ----------------------------
# Main Loop
# ----------------------------

color_history = []

while True:

    hue = read_hue()

    # Debug output
    brain.screen.clear_screen()
    brain.screen.set_cursor(1, 1)
    brain.screen.print("Hue: ")
    brain.screen.print(hue)

    drive_forward()

    if is_red(hue):
        turn_right()
        color_history.append("red")

    elif is_green(hue):
        turn_left()
        color_history.append("green")

    elif is_blue(hue):
        color_history.append("blue")

    elif is_carpet(hue) and len(color_history) > 0:

        last_color = color_history[-1]

        if last_color == "green":
            while not is_green(hue):
                drivetrain.turn_for(LEFT, 5, DEGREES)
                hue = read_hue()

        elif last_color == "red":
            while not is_red(hue):
                drivetrain.turn_for(RIGHT, 5, DEGREES)
                hue = read_hue()

        elif last_color == "blue":
            drivetrain.turn_to_heading(180, DEGREES)
            drivetrain.drive_for(FORWARD, 20, INCHES)
            drivetrain.turn_for(LEFT, 80, DEGREES)
            drivetrain.drive_for(FORWARD, 20, INCHES)

    wait(10, MSEC)
```